

Cryptography: turning a readable note into unreadable noise

Anything you send across a network can be copied by someone in the middle. Cryptography doesn't stop them grabbing it — it makes what they grab useless. This module walks the whole story, from a 2,000-year-old shift cipher to the encryption protecting your bank login right now.

What this key knowledge covers

This summary distils everything you need for **KK10 — Cryptography: turning a readable note into unreadable noise** in Cyber Security. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

01 What cryptography actually is

Cryptography is the practice of transforming readable information into a scrambled form so that only someone with the right secret can read it. It is the tool the study design points to for protecting **confidentiality** of data on a network.

Four words do all the heavy lifting in an exam answer, so get them exact:

Plaintext is the original readable message. The **cipher** (or algorithm) is the *method* of scrambling. The **key** is the secret setting that the method uses. **Ciphertext** is the scrambled output. **Encryption** goes plaintext→ciphertext; **decryption** goes back.

The single most examined idea: security comes from keeping the **key** secret, *not* from keeping the **method** secret. A good cipher stays safe even if an attacker knows exactly how it works — they still can't read anything without the key. This is **Kerckhoffs's principle**, and it's why “we invented our own secret algorithm” is a red flag, not a feature.

Sending plaintext is like writing on a **postcard**: every postie who handles it can read it. Encrypting is like sealing the note in a **locked steel box**. Raven can still grab the box off the network, shake it, photograph it — but without the key, what's inside is noise. The rest of this module is really one question: **how do Maya and her friend both end up holding the right key?**

02 The Caesar cipher

The oldest trick in the book: shift every letter forward by a fixed number of places. It's a **substitution cipher** — each letter is swapped for another — and the key is just the shift amount.

Caesar cipher disk

Drag the shift to rotate the inner ring. The outer ring is the plaintext alphabet; the inner ring is what each letter becomes. Maya's app once used this — watch how few keys there are.

Why this is hopeless for real security: there are only **25 useful keys**. Raven just tries all of them — that's a **brute-force attack**, and on 25 options it takes seconds. Worse, even one guess can be skipped: the most common letter in the ciphertext is almost always plain E, so **frequency analysis** cracks it without trying every key at all.

03 Vigenère & polyalphabetic ciphers

The fix people reached for next: stop using *one* shift for the whole message. A **polyalphabetic** cipher changes the shift letter by letter. The Vigenère cipher does this with a repeating **keyword**.

Each letter of the keyword gives the shift for the matching letter of the message. A =shift 0, B =shift 1, C =shift 2, and so on. The keyword repeats across the whole message.

Vigenère encoder

Type a keyword and watch the same plaintext letter encrypt to *different* ciphertext letters depending on where it sits. That's what kills simple frequency analysis.

Look at the highlighted letters: the same plaintext letter became **two different** ciphertext letters. With a keyword, a simple letter count no longer cracks it.

Polyalphabetic ciphers were considered “unbreakable” for 300 years. They aren't — if the keyword is short and repeats, the pattern eventually leaks (the **Kasiski** attack). In WWII, machine ciphers like Enigma pushed this idea to an extreme, and the codebreakers at **Bletchley Park** — including Alan Turing — broke them anyway. The lesson the study design wants you to take: **a longer, more random key is harder to break**, which is exactly why modern keys are huge random numbers, not English words.

04 Symmetric encryption — one shared key (AES)

Caesar and Vigenère are both **symmetric**: the *same* key encrypts and decrypts. The modern symmetric standard is **AES** (Advanced Encryption Standard). It does to bits what Vigenère does to letters, but vastly more thoroughly and with an enormous key.

AES is **fast**, runs in hardware on every phone and laptop, and with a 256-bit key the number of possible keys is so large that brute force is physically impossible. The key, not the method, carries all the secrecy — AES's algorithm is fully public.

Why key length is the whole game

Each extra bit **doubles** the number of keys. That's why “256-bit” isn't twice as strong as 128-bit — it's 2^{128} times the keyspace. No computer that could ever exist brute-forces AES-256.

The fatal catch — key distribution. AES is brilliant, but it has a chicken-and-egg problem. Before Maya and her friend can send encrypted notes, they both need the *same* secret key.

How does Maya get that key to her friend across the network **without Raven copying it on the way**? You can't encrypt the key... because you'd need a key to do that. This single problem is what asymmetric encryption was invented to solve.

05 Asymmetric encryption — two keys (RSA · ECDH · ECDSA)

The breakthrough: give everyone a **pair** of mathematically linked keys. A **public key** that anyone may have, and a **private key** kept secret. What one key locks, only the *other* can unlock.

Maya hands out **open padlocks** with her name on them (her public key) to anyone who asks — she doesn't care if Raven grabs a hundred of them. To send Maya a private message, you put it in a box and snap one of her padlocks shut. Now **only Maya's private key** can open it — not the sender, not Raven, not even another one of Maya's own padlocks. The lock can be public because **locking and unlocking use different keys**.

- ✓ **Fast** — encrypts large amounts of data with little processing.
- ✓ Mature, hardware-accelerated (AES everywhere).
- - **Key distribution problem** — both sides must already share the secret.
- - Key count explodes — n people need many separate shared keys.
- ✓ **Solves key distribution** — the public key can be shared openly.
- ✓ Enables **digital signatures** — proof of who sent something.
- - **Slow** — far heavier maths, poor for large data.
- - Relies on the private key never leaking.

The three names in the study design

All three are asymmetric, but they're used for different jobs — examiners like you to know *what each one is for*, not just that it exists.

Confidentiality vs authenticity. Encrypting protects **confidentiality** (nobody can read it). Signing protects **authenticity and integrity** (you know who sent it and that it wasn't changed). Don't mix these up in an exam — a signature does *not* hide a message, and encryption alone does *not* prove the sender.

06 Lab — how a real connection is built

Real systems don't choose symmetric *or* asymmetric — they use **both**, each for what it's good at. This is exactly what happens in the half-second before your bank page loads (TLS / HTTPS). Step through it.

The hybrid handshake

Maya's phone (left) wants a private channel to the PigeonPost server (right). Raven sits on the wire between them. Press play and watch which key does which job.

The exam-grade insight: asymmetric is used *once*, only to deliver the symmetric key safely; then fast symmetric encryption protects everything after. You get the **key-distribution fix** of asymmetric *and* the **speed** of symmetric. If you can explain *why* a system uses both, you're answering at the level top responses sit.

07 Key terms — tap to flip

The exact vocabulary examiners reward. Tap a card to check yourself, then say the definition aloud before you flip.

The original, readable message before any encryption is applied.

The scrambled, unreadable output of encryption. Useless without the key.

The algorithm used to scramble. It can be public — secrecy lives in the key, not the method.

The setting the cipher uses. Keeping the key secret is what keeps the message safe.

Same key encrypts and decrypts. Fast, but both sides must already share the secret. e.g. AES.

A linked key pair: what one key locks, only the other unlocks. Solves key distribution.

Advanced Encryption Standard. Modern, fast, hardware-accelerated, with huge keys (128/256-bit).

10 Writing trainer — C·M·E

Crypto questions reward a tight three-move structure. Use it for any “explain / justify” cryptography answer.

Name the technique and what it protects (confidentiality / integrity / authenticity).

Say *how* it works in one line — keys, public/private, shift, etc.

Link it to the scenario: the benefit or trade-off *here*.

Prompt: Maya wants to send a private message to one friend over the school Wi-Fi. Explain how asymmetric encryption keeps the message confidential even though Raven can see everything on the network. (3 marks)

Maya's friend creates a public/private key pair and shares the public key openly — it doesn't matter that Raven copies it (Claim: confidentiality via asymmetric encryption). Maya encrypts the message with that public key, and because of the way the key pair is mathematically linked, the ciphertext can only be decrypted by the matching private key, which never leaves her friend's device (Mechanism: different keys lock and unlock). So although Raven intercepts the ciphertext on the shared Wi-Fi, she has no private key and the message stays unreadable to her (Evaluation: tied to the scenario).

- * 1 mark: plaintext = the original/readable message.
- * 1 mark: ciphertext = the scrambled/encrypted, unreadable output (ideally noting it needs the key to reverse).
- * 1 mark: identifies the very small keyspace (only 25 keys / a single shift).
- * 1 mark: explains brute force — an attacker tries all keys quickly.
- * 1 mark: links to why that means no real security (or cites frequency analysis).

Common traps & misconceptions

“Encryption proves who sent the message.”

! ► **The slip:** It doesn't. Encryption protects **confidentiality** — it hides the *content*. **The fix:** Proving the sender is **authenticity**, which comes from a **digital signature** (e.g. ECDSA), not from encryption.

“A cipher is secure as long as nobody knows the algorithm.”

! ► **The slip:** This is ‘security through obscurity’ and it fails — algorithms leak, get reverse-engineered, or are public anyway. **The fix:** Per **Kerckhoffs's principle**, security must rest on the secret **key**. A home-made secret algorithm is a warning sign, not strength.

“The public and private key are basically the same / interchangeable.”

! ► **The slip:** They're a mathematically linked **pair**, but not interchangeable. **The fix:** What the **public** key locks, only the **private** key unlocks. The public key cannot decrypt what it just encrypted — that's exactly why it's safe to share.

“Asymmetric encryption is better, so it replaced symmetric.”

! ► **The slip:** Asymmetric is far **slower** and poor for large amounts of data. **The fix:** Real systems use **hybrid** encryption: asymmetric just to share a key, then fast **symmetric (AES)** for the actual traffic. Both are needed.

“256-bit encryption is twice as strong as 128-bit.”

! ► **The slip:** It sounds like simple doubling, but key strength is exponential. **The fix:** Each extra bit **doubles** the keyspace, so AES-256 has 2^{128} times as many keys as AES-128 — not double.

“Vigenère is unbreakable because it uses a keyword.”

! ► **The slip:** It resists *simple* frequency analysis, but it isn't unbreakable. **The fix:** A short, repeating keyword leaks a pattern that the **Kasiski** method exposes. The takeaway: **longer, more random keys are stronger** — which is why modern keys aren't words.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Define · 2 marks

Define the terms plaintext and ciphertext.

✗ A weak answer

er response Plaintext is normal text and ciphertext is secret text you can't read. Too loose. "Secret text" doesn't show the relationship, and there's no mention of encryption linking them.

✓ A strong answer

er response Plaintext is the original, readable message before encryption. Ciphertext is the scrambled output produced when a cipher and key encrypt the plaintext, and it is unreadable without the correct key.

How the marks are awarded

- 1 mark: plaintext = the original/readable message.
- 1 mark: ciphertext = the scrambled/encrypted, unreadable output (ideally noting it needs the key to reverse).

Q2. Explain · 3 marks

Explain why the Caesar cipher is unsuitable for protecting data sent over a real network.

✗ A weak answer

er response Because it's really old and everyone knows it, so it's not safe any more. Age and familiarity aren't the real reason. No mention of the tiny keyspace or how an attacker breaks it.

✓ A strong answer

er response The Caesar cipher only has 25 usable keys, because the key is just a shift from 1 to 25. An attacker on the network can simply try every shift — a brute-force attack — and read the message in seconds. Frequency analysis makes it even faster, since the most common ciphertext letter is almost always E. With so few keys, it gives no real protection.

How the marks are awarded

- 1 mark: identifies the very small keyspace (only 25 keys / a single shift).
- 1 mark: explains brute force — an attacker tries all keys quickly.
- 1 mark: links to why that means no real security (or cites frequency analysis).

Q3. Compare · 4 marks

Compare symmetric and asymmetric encryption. Refer to one advantage and one disadvantage of each.

✗ A weak answer

er response Symmetric uses one key and asymmetric uses two keys. Asymmetric is more secure so it's better. States a difference but gives no real advantages/disadvantages, and “more secure so better” is wrong — the trade-off is speed vs key distribution.

✓ A strong answer

er response Symmetric encryption uses one shared key for both encryption and decryption (e.g. AES). Its advantage is speed — it can protect large amounts of data efficiently — but its disadvantage is the key distribution problem: both parties must already share the secret key safely. Asymmetric encryption uses a public/private key pair. Its advantage is that the public key can be shared openly, solving key distribution and enabling signatures, but its disadvantage is that it is much slower and unsuited to large amounts of data.

How the marks are awarded

- 1 mark: symmetric = one shared key; asymmetric = public/private pair.
- 1 mark: a valid advantage of each (symmetric: fast; asymmetric: solves key distribution / open sharing).
- 1 mark: a valid disadvantage of each (symmetric: key distribution; asymmetric: slow).
- 1 mark: genuine comparison (the trade-off is speed vs safe key sharing), not just two separate descriptions.

Q4. Justify · 4 marks

A messaging app uses asymmetric encryption only to exchange a key, then switches to AES for the whole conversation. Justify this hybrid design.

✗ A weak answer

er response Because using both is more secure than using just one type of encryption. Asserts “more secure” without explaining the reasoning. Doesn't say what each part contributes.

✓ A strong answer

er response Asymmetric encryption is the only one of the two that can safely deliver a key across an open network, because the public key can be shared without an eavesdropper being able to decrypt what it locks. However, asymmetric is slow, so it would be inefficient for an entire conversation. Once the shared AES key has been exchanged, symmetric AES is used because it is fast and can encrypt large amounts of data efficiently. The hybrid design therefore gets the key-distribution benefit of asymmetric and the speed of symmetric — each is used only for what it does best.

How the marks are awarded

- 1 mark: asymmetric solves safe key exchange over an open channel.
- 1 mark: recognises asymmetric is too slow for bulk data.
- 1 mark: AES (symmetric) is fast/efficient for the conversation data.
- 1 mark: clear justification — the design combines the strengths of both / each used for its best job.